Índice

Objetivos de la sesión
Conocimientos y materiales necesarios
1. Introducción al simulador pyMIPS64
2. Ensamblador del simulador pyMIPS64
3. Configuración de la microarquitectura
4. Simulación de un programa complejo
Ejercicios
Apéndice. Juego de instrucciones del simulador pyMIPS64

Introducción a la arquitectura MIPS64

Área de Arquitectura y Tecnología de Computadores – Versión 3.0.12009, 25/09/2023

Objetivos de la sesión

En esta sesión se introduce el simulador pyMIPS64 y se utilizará para analizar la microarquitectura segmentada en 5 etapas de la CPU MIPS64 presentada en la parte teórica de la asignatura. Los principales objetivos de esta sesión son:

- Familiarizarse con el uso del simulador pyMIPS64.
- Introducir el ensamblador empleado por el simulador.
- Comprender el funcionamiento de programas escritos en ensamblador para MIPS64.

Conocimientos y materiales necesarios

Para poder realizar esta sesión, el alumno debe:

- Revisar la teoría correspondiente al tema dedicado a la CPU. Es importante tener claras las ideas teóricas sobre la arquitectura del juego de instrucciones MIPS64.
- Durante la sesión se plantearán una serie de preguntas que deben responderse en el correspondiente cuestionario en el campus virtual. Puede abrirse el cuestionario en otra pestaña del navegador pinchando en el enlace mientras se mantiene pulsada la tecla `Ctrl`.

Para llevar a cabo la práctica es necesario disponer del simulador pyMIPS64, que actualmente corre sólo en Windows, por lo que es necesaria una máquina con dicho sistema operativo.

---

1. Introducción al simulador pyMIPS64

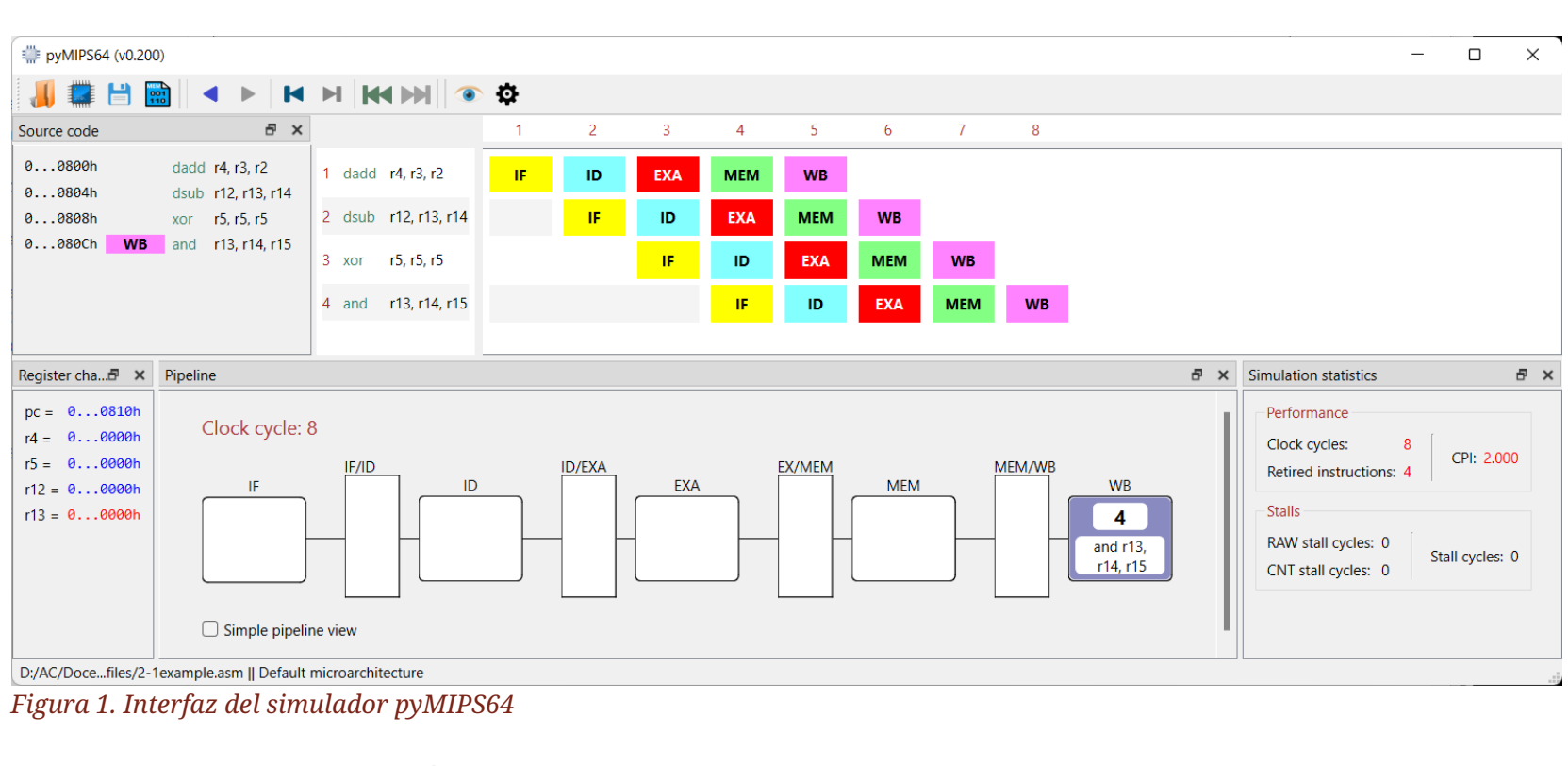
Como has visto en las clases teóricas, MIPS64 es una arquitectura RISC de 64 bits con un juego de instrucciones muy simple. En las prácticas de la asignatura vamos a utilizar el simulador pyMIPS64, que permite simular la ejecución de programas escritos en ensamblador para MIPS64, para estudiar el comportamiento del *pipeline* de la CPU estudiada en las clases teóricas.

- Descarga el simulador pyMIPS64 en tu equipo.
- No es necesario instalarlo, basta con descomprimir el archivo `pyMIPS64.zip` descargado. Dentro la carpeta `pymips64` descomprimida encontrarás un archivo `pyMIPS64.exe` con el ejecutable de la aplicación.
- Para facilitar el uso del simulador crea un acceso directo a dicho archivo ejecutable y muévelo al escritorio. De forma opcional, se le puede asociar el icono de la aplicación haciendo click con el botón derecho del ratón sobre el acceso directo, seleccionando **Propiedades › Cambiar icono** y eligiendo como icono el que se encuentra en el directorio `pyMIPS64\Icons`.
- Para probar el simulador son necesarios los archivos de esta sesión que debes descargar del enlace archivos y descomprimir en tu directorio de trabajo.

Con esto, ya estamos en disposición de probar el simulador.

- Ejecuta el simulador utilizando el acceso directo. Durante el arranque automáticamente se comprueba si existe alguna actualización. En ese caso se recomienda aceptar la actualización, para lo cual debes cerrar la ventana actual del simulador y pulsar el botón [**Aceptar**] en el diálogo.
- Abre el archivo de ejemplo `2-1example.asm` pinchando sobre el icono 📂 de la barra de herramientas.
- Realiza una simulación completa pichando sobre el icono ⏭. El aspecto del simulador es el mostrado en la figura 1 tras ajustar el tamaño de los paneles de la forma habitual, pinchando con el botón izquierdo del ratón sobre la separación entre ellos y arrastrando.

A continuación se describen los elementos de la interfaz gráfica del simulador mostrados en la figura 1.



```
0...0800h      dadd  r4, r3, r2
0...0804h      dsub  r12, r13, r14
0...0808h      xor   r5, r5, r5
0...080Ch  WB  and   r13, r14, r15
```

```
               1    2    3    4    5    6    7    8
1 dadd r4, r3, r2      IF   ID   EXA  MEM  WB
2 dsub r12, r13, r14        IF   ID   EXA  MEM  WB
3 xor r5, r5, r5                 IF   ID   EXA  MEM  WB
4 and r13, r14, r15                   IF   ID   EXA  MEM  WB
```

```
pc = 0...0810h
r4 = 0...0000h
r5 = 0...0000h
r12 = 0...0000h
r13 = 0...0000h
```

Clock cycle: 8

IF — IF/ID — ID — ID/EXA — EXA — EX/MEM — MEM — MEM/WB — WB: 4 and r13, r14, r15

Performance: Clock cycles: 8; Retired instructions: 4; CPI: 2.000
Stalls: RAW stall cycles: 0; CNT stall cycles: 0; Stall cycles: 0

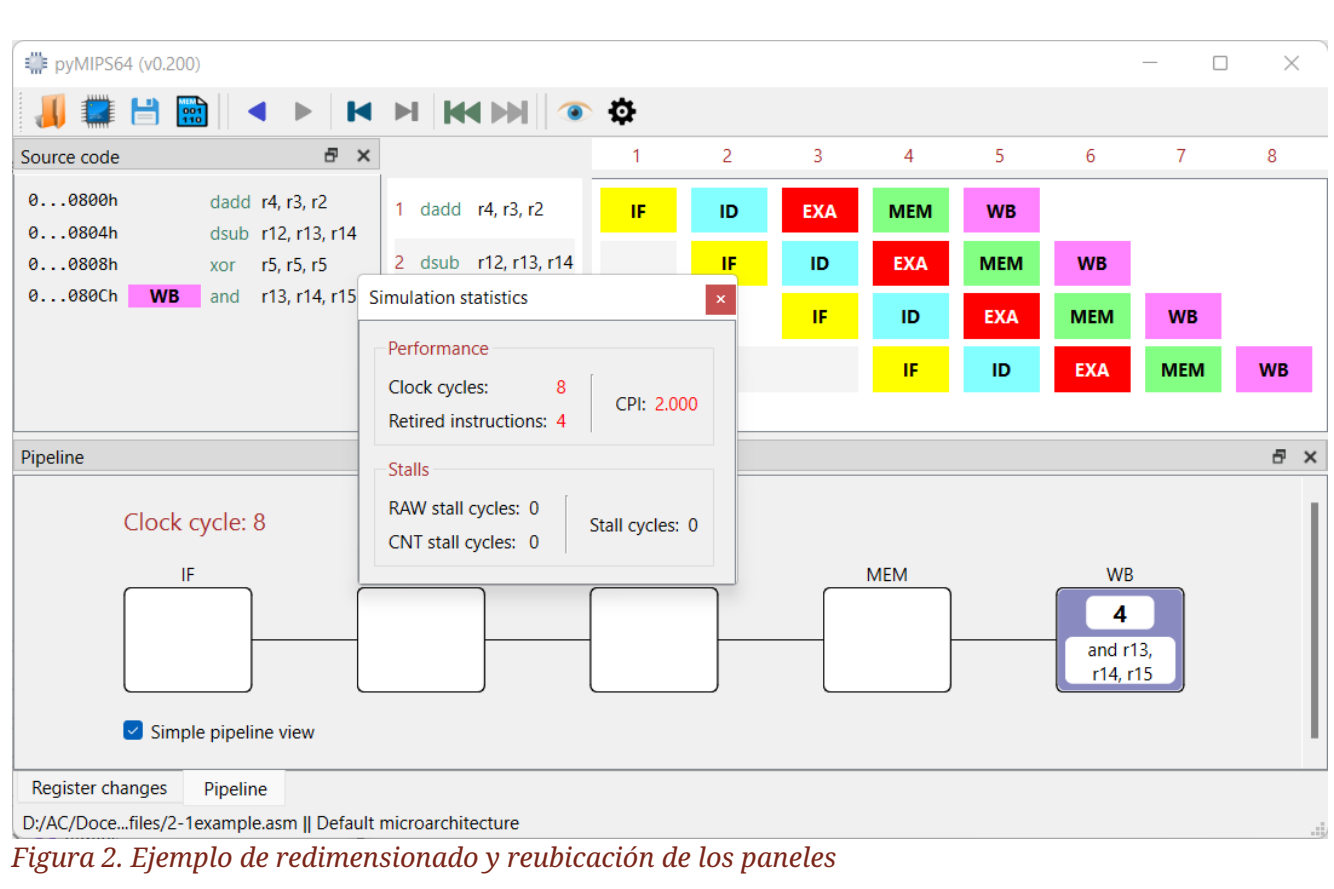
D:/AC/Doce...files/2-1example.asm || Default microarchitecture

Figura 1. Interfaz del simulador pyMIPS64

En primer lugar, se observan un área central y cuatro paneles, aunque dependiendo del archivo a simular y la configuración del simulador pueden aparecer algunos más:

- **Source code**. Muestra la sección de código del programa junto con la dirección a partir de la que se almacena cada instrucción. Las instrucciones se cargan en memoria a partir de la dirección 800h. Debe tenerse en cuenta que las direcciones de memoria son de 64 bits, por lo que requieren 16 dígitos hexadecimales. Para abreviar, la secuencia `0...0` indica una secuencia de dígitos todo ceros. Durante la simulación se muestra además la etapa que ha completado la instrucción en el ciclo actual. Por ejemplo, al final de la simulación la última instrucción del programa ha terminado la etapa `WB`.
- **Register changes**. Permite observar el estado de los registros durante la simulación. Puesto que en la mayor parte de nuestros programas sólo se modifican unos pocos registros, resulta conveniente mostrar sólo los registros que han sido escritos, por lo que aquellos que no aparecen permanecen en el estado inicial con todos sus bits a cero. Cuando uno de los registros mostrados aparece en rojo significa que ha sido escrito en el último ciclo de reloj, justo lo que hace la instrucción `and r13, r14, r15` sobre el registro `r13` en el último ciclo de reloj.
- **Pipeline**. Muestra el estado del cauce en cada ciclo de reloj. Dentro del cauce pueden observarse tanto las etapas como los registros de segmentación entre ellas. Dependiendo de la microarquitectura empleada pueden aparecer más etapas y registros de segmentación. Al final de la ejecución del programa todas las etapas excepto la `WB` aparecen en blanco, indicando que están inactivas. La única activa es la `WB`, en la que se muestra el índice de la instrucción procesada, la 4, y el mnemónico correspondiente. Si se desea simplificar la visualización del cauce pueden ocultarse los registros de segmentación marcando la casilla `Simple pipeline view`.
- **Simulation statistics**. Proporciona estadísticas sobre la simulación. En la figura se muestran estadísticas de rendimiento (*performance*) y sobre detenciones (*stalls*). El tipo de información mostrada depende del archivo ensamblador simulado y de la configuración del simulador. Por ejemplo, en la figura se indica que se han simulado 8 ciclos, se han retirado (terminado) 4 instrucciones y que por lo tanto el CPI (Ciclos Por Instrucción) es 8/4 = 2.
- Área principal. Situada en la parte superior derecha de la figura, no tiene barra de título ni controles de ventana. Muestra la evolución del *pipeline* durante la ejecución del programa. Tiene tres partes:
    - Instrucciones ejecutadas. Aparecen numeradas en la parte izquierda.
    - Ciclos de reloj. Aparecen numerados en la parte superior.
    - Cronograma de etapas. Aparece en la parte central.

Los paneles se pueden hacer flotantes pinchando sobre su icono 🗗, independizándose de la ventana principal. También se pueden arrastrar para situarlos en otra zona de la ventana principal, o incluso solaparlos con otros paneles creando pestañas. La figura 2 muestra un ejemplo.



```
0...0800h      dadd  r4, r3, r2
0...0804h      dsub  r12, r13, r14
0...0808h      xor   r5, r5, r5
0...080Ch  WB  and   r13, r14, r15
```

Performance: Clock cycles: 8; Retired instructions: 4; CPI: 2.000
Stalls: RAW stall cycles: 0; CNT stall cycles: 0; Stall cycles: 0

Clock cycle: 8

☑ Simple pipeline view

D:/AC/Doce...files/2-1example.asm || Default microarchitecture

Figura 2. Ejemplo de redimensionado y reubicación de los paneles

En la parte superior de la interfaz se encuentra una barra de herramientas. Si acercas el puntero del ratón a cualquiera de los iconos de la misma aparece una descripción emergente que incluye el atajo de teclado asociado. A continuación se describen brevemente:

- 📂 Permite seleccionar un archivo fuente ensamblador o un archivo de traza. En el próximo apartado se describe la sintaxis de los archivos fuente. Los archivos de traza se utilizarán en las sesiones del tema 3. Observa en la figura anterior cómo el nombre del archivo abierto aparece en la barra de estado en la parte inferior de la interfaz.
- ▣ Abre una ventana de diálogo en la cual se puede configurar la microarquitectura a simular. En las clases de teoría se presentan diferentes alternativas que pueden estudiarse con la ayuda del simulador.
- 💾 Proporciona la posibilidad de guardar el archivo ensamblador cargado junto con la configuración actual de la microarquitectura. De esta forma, cuando se cargue de nuevo el archivo ensamblador, no sólo se cargará el código fuente, sino que se configurará automáticamente la microarquitectura.
- ▤ Permite generar una traza con todos los accesos a memoria del programa y guardarlos en un archivo. De momento no emplearemos esta opción hasta que lleguemos a las prácticas de caché.
- ◀ Retrocede un ciclo de reloj en la simulación.
- ▶ Avanza un ciclo de reloj en la simulación.
- ⏮ Retrocede el número de ciclos de reloj necesarios hasta la terminación de una instrucción, es decir, justo hasta que una instrucción completó la etapa `WB`.
- ⏭ Avanza el número de ciclos de reloj necesarios hasta la terminación de una instrucción, es decir, justo hasta que una nueva instrucción completa la etapa `WB`.
- ⏪ Retrocede la simulación hasta el principio.
- ⏩ Avanza la simulation hasta el final.
- 👁 Permite seleccionar los paneles que se muestran y se ocultan. Se pueden ocultar todos los paneles, pero no el área principal que muestra el cronograma del *pipeline*. Los paneles también se pueden ocultar pinchando sobre el aspa en la esquina superior derecha del panel.
- ⚙ Muestra los créditos del simulador y permite mostrar/ocultar información adicional en los cronogramas del *pipeline* (si está disponible).

- Con el archivo de ejemplo `2-1example.asm` abierto, prueba la funcionalidad descrita.

---

2. Ensamblador del simulador pyMIPS64

Los archivos ensamblador empleados por el simulador tienen extensión `.asm` y definen el programa a simular. Las instrucciones soportadas son básicamente el subconjunto de instrucciones MIPS64 empleado en las clases de teoría, mostrado en el apéndice.

Un programa en el ensamblador del simulador pyMIPS64 tiene hasta tres secciones:

- Sección de código. Definida a partir de la directiva `.code`. Esta sección incluye las instrucciones del programa y es opcional. Se recomienda identar el código de tal forma que se liberen las primeras columnas para definir etiquetas como destinos de las instrucciones de salto. El programa empezará a ejecutarse por la primera instrucción de la sección de código.
- Sección de datos. Definida a partir de la directiva `.data`. Esta sección inlcuye las definiciones de variables del programa y también es opcional.
- Sección de microarquitectura. Definida a partir de la directiva `.microarch`. Esta sección es opcional y la emplearemos preferentemente en los exámenes. Contiene la definición de la microarquitectura del simulador empleada exclusivamente en la simulación del archivo fuente. Una vez se abre otro archivo fuente esa microarquitectura se pierde y se simula con la microarquitectura original.

Todas aquellas líneas que aparezcan antes de cualquiera de las directivas anteriores se supone pertenecen a la sección de código, por lo que es posible un archivo fuente con sólo instrucciones y sin directivas. Si editas el archivo `2-1example.asm` verás que ese es su caso.

La sección de datos puede definir variables enteras de tamaño 8 bits con la directiva `byte`, enteras de 16 bits con la directiva `hword`, enteras de 32 bits con la directiva `word` y 64 bits con la directiva `dword` para valores enteros, o `double` para valores reales. Para definir *arrays* de enteros o reales los valores se separan por comas. Incluye además una forma abreviada de definir *arrays* añadiendo el sufijo `n` a las directivas anteriores junto con dos valores; el primero indica el número de elementos y el segundo el valor a almacenar. Por ejemplo, `wordn 20, 10h` se corresponde con un *array* de 20 palabras inicializadas a `10h`.

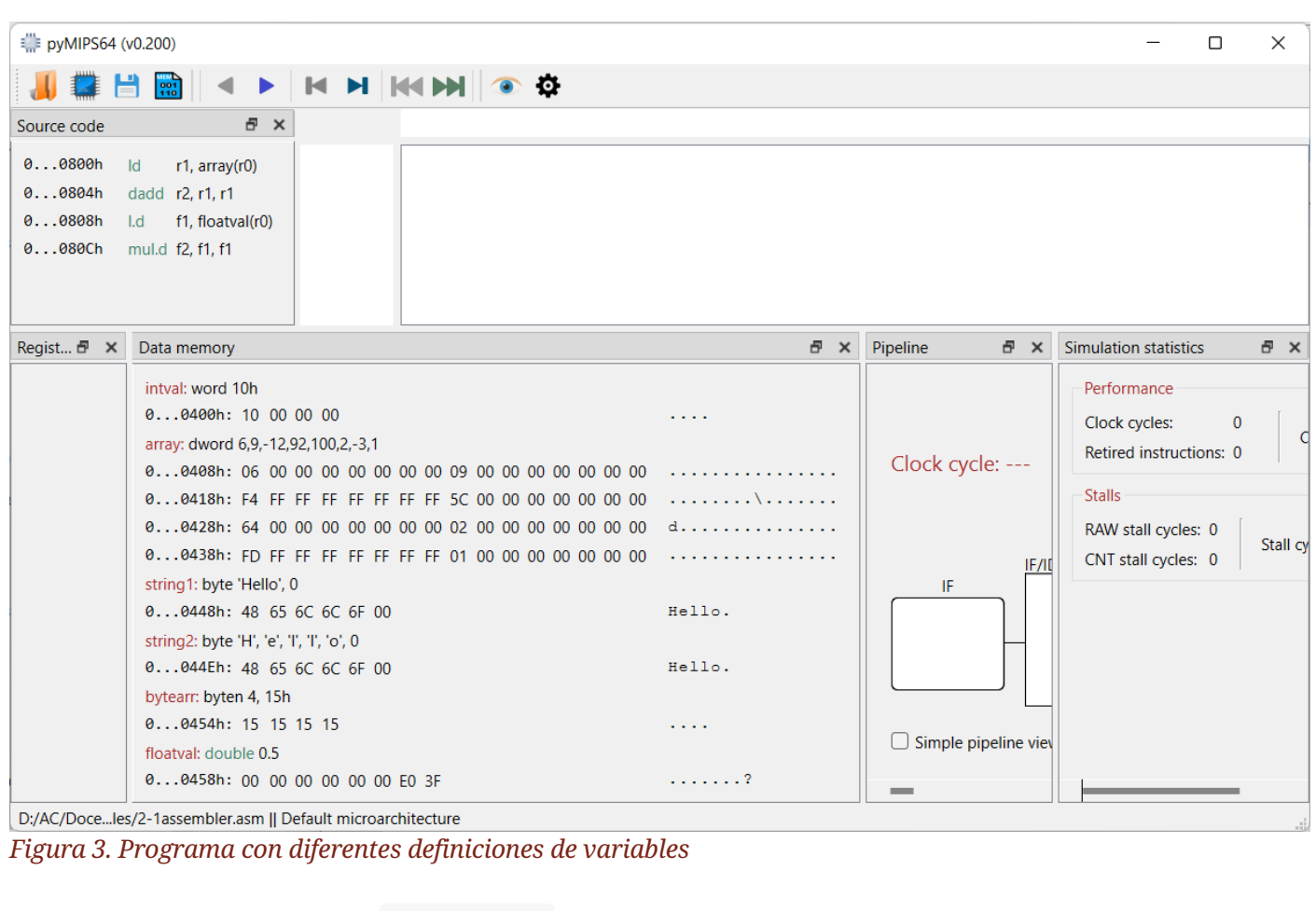
Los valores enteros se expresan por defecto en decimal, aunque también es posible expresarlos en hexadecimal añadiendo el sufijo `h`. Opcionalmente pueden llevar un signo `-` para los enteros negativos. En el caso de los enteros de tipo `byte` que deban almacenar el código ASCII de un carácter es posible definirlos de la forma habitual, con el carácter entre comillas simples. Si se escribe más de un carácter entre las comillas simples se define una cadena de caracteres que no incluye el carácter nulo de terminación.

A continuación se muestra un ejemplo de programa que incluye muchas de las características descritas.

```
.data
intval:   word 10h                        ; Decimal integer 16
array:    dword 6,9,-12,92,100,2,-3,1     ; Array of 64-bit integer numbers
string1:  byte 'Hello', 0                 ; A null terminated string
string2:  byte 'H', 'e', 'l', 'l', 'o', 0 ; The same string
bytearr:  byten 4, 15h                    ; The same as byte 15h, 15h, 15h, 15h
floatval: double 0.5                      ; IEEE-754 double value

.code
ld    r1, array(r0)    ; r1 = array[0] = 6
dadd  r2, r1, r1       ; r2 = r1+r1 = 12
l.d   f1, floatval(r0) ; f1 = 0.5
mul.d f2, f1, f1       ; f2 = f1*f1 = 0.25
```

- Abre el archivo fuente `2-1assembler.asm` en el simulador. La figura 3 muestra el aspecto del simulador tras cargar el fichero.



```
0...0800h  ld     r1, array(r0)
0...0804h  dadd   r2, r1, r1
0...0808h  l.d    f1, floatval(r0)
0...080Ch  mul.d  f2, f1, f1
```

```
intval: word 10h
0...0400h: 10 00 00 00                                      ....
array: dword 6,9,-12,92,100,2,-3,1
0...0408h: 06 00 00 00 00 00 00 00 09 00 00 00 00 00 00 00 ................
0...0418h: F4 FF FF FF FF FF FF FF 5C 00 00 00 00 00 00 00 ........\.......
0...0428h: 64 00 00 00 00 00 00 00 02 00 00 00 00 00 00 00 d...............
0...0438h: FD FF FF FF FF FF FF FF 01 00 00 00 00 00 00 00 ................
string1: byte 'Hello', 0
0...0448h: 48 65 6C 6C 6F 00                                Hello.
string2: byte 'H', 'e', 'l', 'l', 'o', 0
0...044Eh: 48 65 6C 6C 6F 00                                Hello.
bytearr: byten 4, 15h
0...0454h: 15 15 15 15                                      ....
floatval: double 0.5
0...0458h: 00 00 00 00 00 00 E0 3F                          .......?
```

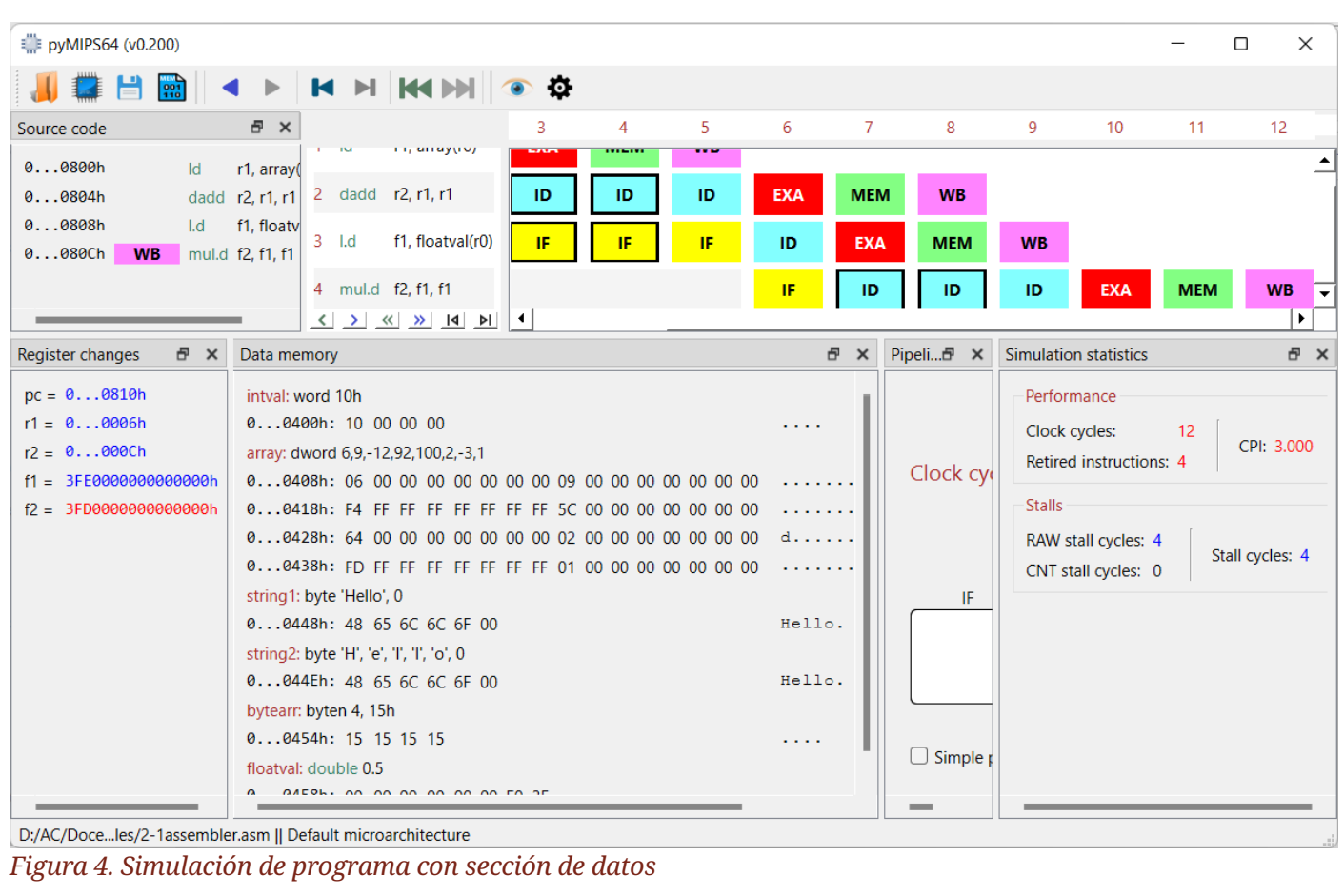
Clock cycle: ---

Performance: Clock cycles: 0; Retired instructions: 0
Stalls: RAW stall cycles: 0; CNT stall cycles: 0

D:/AC/Doce...les/2-1assembler.asm || Default microarchitecture

Figura 3. Programa con diferentes definiciones de variables

- Observa por un lado el panel `Data memory` con los datos de memoria. Los datos se almacenan en memoria en formato *little endian* a partir de la dirección de memoria 400h. En la parte derecha del panel se muestra el contenido de la memoria interpretando los bytes como caracteres ASCII, lo cual resulta útil con programas que procesan cadenas.
- Observa por otro lado el panel `Source code`, el cual muestra la posición de las instrucciones en memoria a partir de la dirección 800h.
- La variable `intval` tiene un tamaño de 32 bits al estar definida con la directiva `word`, es decir, 4 bytes. La variable se almacena a partir de la dirección de memoria 400h. Recuerda que estas direcciones son de 64 bits pero para abreviar no representamos los dígitos más significativos a cero. ¿Cuál es la dirección más alta que ocupa en memoria la variable `intval`? Responde en el cuestionario: pregunta 1.
- El primer elemento de la variable `array` comienza en la dirección 408h. ¿Por qué no ocupa la dirección siguiente a la última de la variable `intval`? Responde en el cuestionario: pregunta 2.
- Simula hasta el final del programa. El simulador tendrá un aspecto similar al de la figura 4.



```
0...0800h      ld     r1, array(
0...0804h      dadd   r2, r1, r1
0...0808h      l.d    f1, floatv
0...080Ch  WB  mul.d  f2, f1, f1
```

```
                         3    4    5    6    7    8    9    10   11   12
2 dadd r2, r1, r1        ID   ID   ID   EXA  MEM  WB
3 l.d f1, floatval(r0)   IF   IF   IF   ID   EXA  MEM  WB
4 mul.d f2, f1, f1                      IF   ID   ID   ID   EXA  MEM  WB
```

```
pc = 0...0810h
r1 = 0...0006h
r2 = 0...000Ch
f1 = 3FE0000000000000h
f2 = 3FD0000000000000h
```

Performance: Clock cycles: 12; Retired instructions: 4; CPI: 3.000
Stalls: RAW stall cycles: 4; CNT stall cycles: 0; Stall cycles: 4

D:/AC/Doce...les/2-1assembler.asm || Default microarchitecture

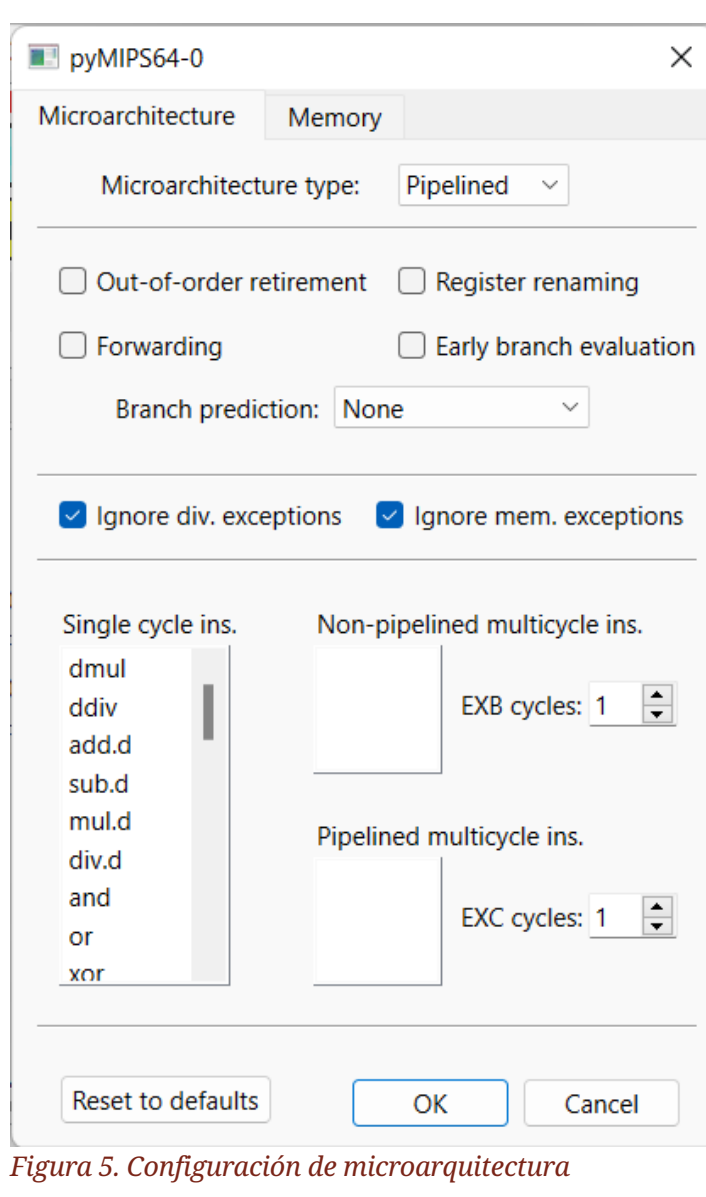
Figura 4. Simulación de programa con sección de datos

---

3. Configuración de la microarquitectura

En las clases expositivas se presentan diferentes alternativas microarquitectónicas con el objetivo de estudiar la mejora del rendimiento de la CPU. El simulador permite configurar la microarquitectura con la que que se simula la ejecución del programa. Por ejemplo, por defecto la unidad de ejecución que se emplea es monociclo y se denomina `EXA`. No obstante, es posible configurar instrucciones para que se ejecuten sobre las unidades multiciclo `EXB` y `EXC`. La unidad de ejecución monociclo, `EXA`, aparece en el cronograma del *pipeline* de la figura anterior.

- Pincha sobre el icono ▣ de la barra de herramientas para abrir el diálogo de configuración de la microarquitectura. Aparece una ventana con dos pestañas como la mostrada en la figura 5. Por defecto la pestaña seleccionada es la de la microarquitectura. La otra pestaña, dedicada a la configuración de memoria sera ignorada de momento.



Microarchitecture type: Pipelined

☐ Out-of-order retirement ☐ Register renaming
☐ Forwarding ☐ Early branch evaluation
Branch prediction: None

☑ Ignore div. exceptions ☑ Ignore mem. exceptions

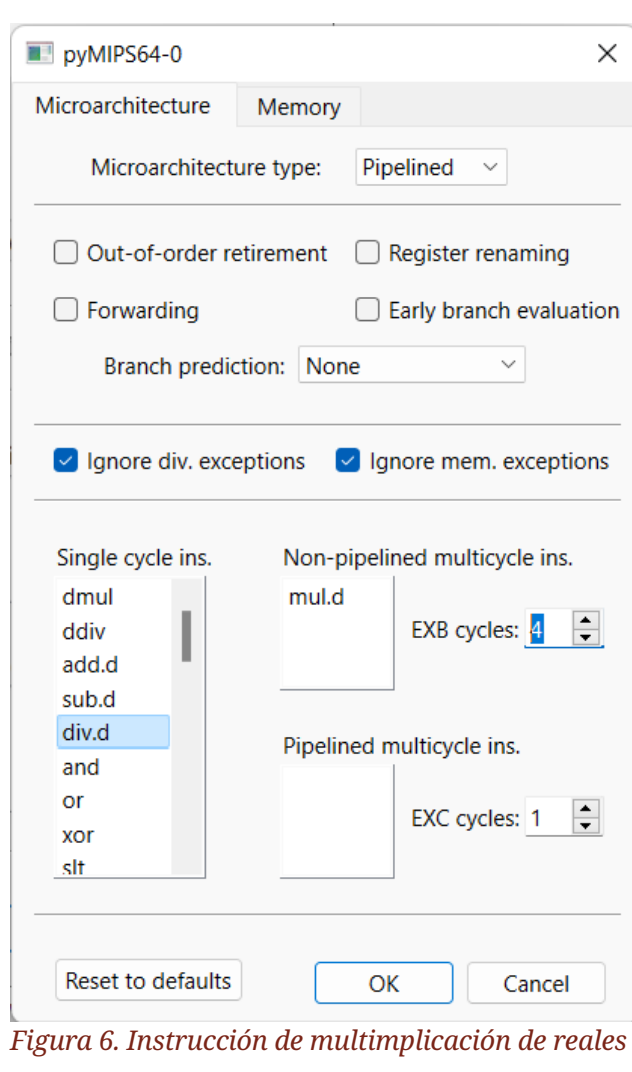
Single cycle ins.: dmul, ddiv, add.d, sub.d, mul.d, div.d, and, or, xor
Non-pipelined multicycle ins. — EXB cycles: 1
Pipelined multicycle ins. — EXC cycles: 1

Figura 5. Configuración de microarquitectura

La configuración con la mayor parte de las opciones desactivadas se corresponde con la microarquitectura básica, es decir, sin ningún tipo de mejora. Sólo aparecen dos casillas activas, que sirven para que el simulador no genere excepciones y termine la simulación cuando se producen divisiones por cero o accesos inválidos a la memoria de datos.

El botón [**Reset to defaults**] restituye el estado por defecto de todas las opciones.

- Ahora vamos a cambiar la microarquitectura para que la instrucción `mul.d` se ejecute en una unidad multiciclo no segmentada de 4 ciclos de reloj. Pincha sobre la instrucción `mul.d` dentro del cuadro `Single cycle ins` y arrástrala hasta el cuadro `Non-pipelined multicycle ins`. Cambia además el número de ciclos de la unidad de ejecución multiciclo no segmentada, `EXB`, para que sea 4 ciclos. El diálogo de configuración tendrá el aspecto de la figura 6. Pulsa a continuación el botón [**OK**] para guardar la nueva microarquitectura.



Microarchitecture type: Pipelined

☐ Out-of-order retirement ☐ Register renaming
☐ Forwarding ☐ Early branch evaluation
Branch prediction: None

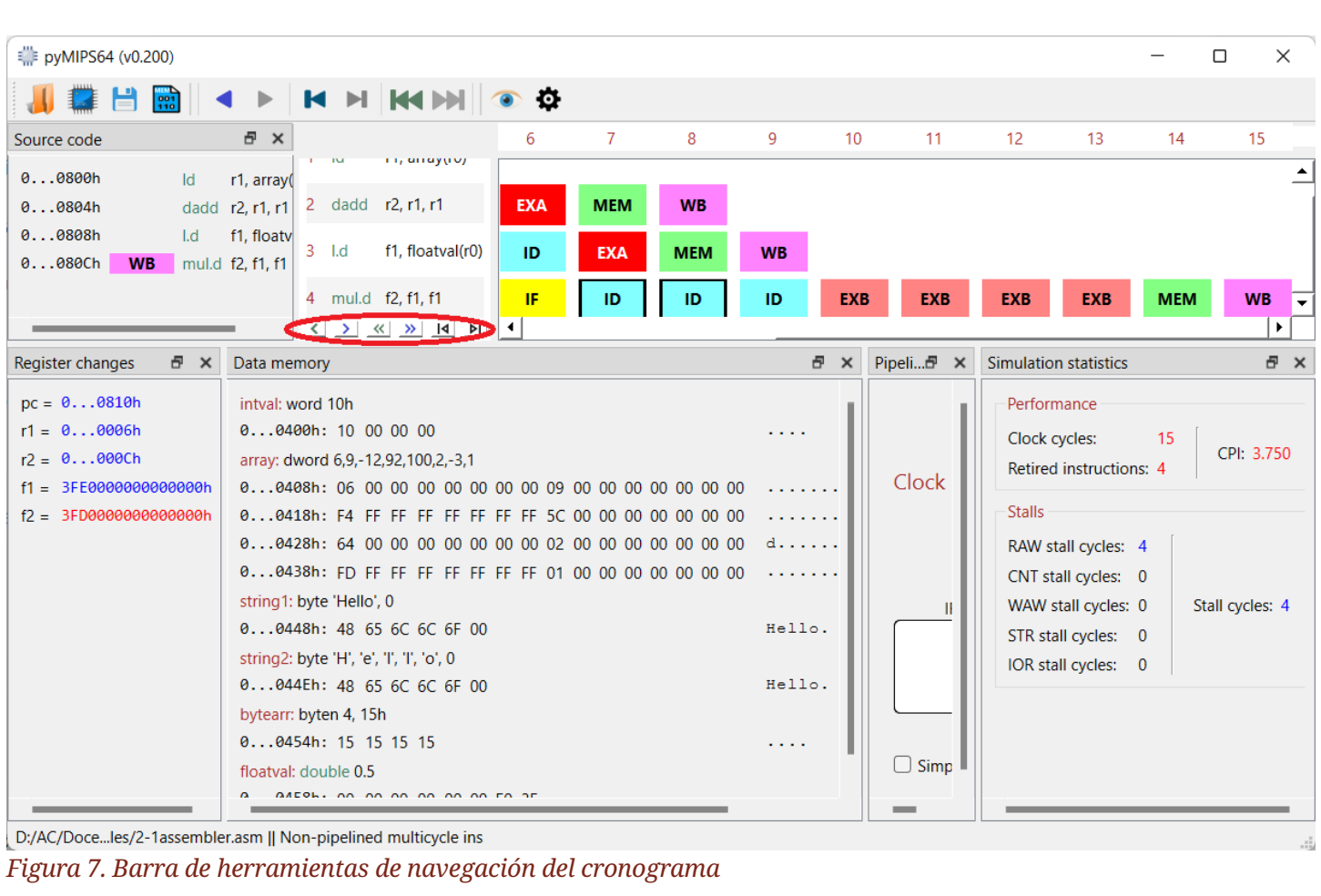
☑ Ignore div. exceptions ☑ Ignore mem. exceptions

Single cycle ins.: dmul, ddiv, add.d, sub.d, div.d, and, or, xor, slt
Non-pipelined multicycle ins.: mul.d — EXB cycles: 4
Pipelined multicycle ins. — EXC cycles: 1

Figura 6. Instrucción de multimplicación de reales multiciclo

- Simula hasta el final y observa el uso de la unidad multiciclo `EXB` para la realización de la multiplicación de reales.

Cuando el cronograma tiene un tamaño que no cabe en el panel correspondiente, además de las barras de desplazamiento correspondientes, aparece una pequeña barra de herramientas debajo de las instrucciones del cronograma. Aparece resaltada en la figura 7.



```
0...0800h      ld     r1, array(
0...0804h      dadd   r2, r1, r1
0...0808h      l.d    f1, floatv
0...080Ch  WB  mul.d  f2, f1, f1
```

```
                         6    7    8    9    10   11   12   13   14   15
2 dadd r2, r1, r1        EXA  MEM  WB
3 l.d f1, floatval(r0)   ID   EXA  MEM  WB
4 mul.d f2, f1, f1       IF   ID   ID   ID   EXB  EXB  EXB  EXB  MEM  WB
```

```
pc = 0...0810h
r1 = 0...0006h
r2 = 0...000Ch
f1 = 3FE0000000000000h
f2 = 3FD0000000000000h
```

Performance: Clock cycles: 15; Retired instructions: 4; CPI: 3.750
Stalls: RAW stall cycles: 4; CNT stall cycles: 0; WAW stall cycles: 0; STR stall cycles: 0; IOR stall cycles: 0; Stall cycles: 4

D:/AC/Doce...les/2-1assembler.asm || Non-pipelined multicycle ins

Figura 7. Barra de herramientas de navegación del cronograma

Con esta barra se puede navegar de una forma más rápida y cómoda que usando las barras de desplazamiento. Es particularmente útil en el caso de cronogramas muy grandes. Tiene estos controles:

- › Mueve un ciclo de reloj adelante con desplazamiento vertical automático.
- » Mueve 10 ciclos de reloj adelante con desplazamiento vertical automático.
- ▷| Desplazamiento horizontal y vertical hasta el final del cronograma.
- ‹ Mueve un ciclo de reloj atrás con desplazamiento vertical automático.
- « Mueve 10 ciclos de reloj atrás con desplazamiento vertical automático.
- |◁ Desplazamiento horizontal y vertical hasta el inicio del cronograma.

---

4. Simulación de un programa complejo

En este apartado se trabajará con un programa que calcula el máximo de una lista de números enteros. Puedes ver su código a continuación.

```
.microarch
    out_of_order_retirement    = true
    forwarding                 = false
    early_branch_evaluation    = true
    register_renaming          = false
    branch_prediction          = always_not_taken
    exb_no_pipelined_cycles    = 1
    exb_instructions           = []
    exc_pipelined_cycles       = 1
    exc_instructions           = []
    ignore_div_exceptions      = false
    ignore_data_mem_exceptions = false

.data
    array: dword 6,9,12,92,100,2,3,1 ; array of numbers
    count: dword 8                   ; item count
    max:   dword 0                   ; maximum

.code
main:
    xor    r8, r8, r8     ; temporary maximum (0 is the minimum)
    xor    r9, r9, r9     ; memory index of the current item
    ld     r10, count(r0) ; remaining items
loop:
    ld     r11, array(r9)  ; current item
    daddi  r10, r10, -1    ; count--
    daddi  r9, r9, 8       ; move index to next item (64 bits)
    slt    r2, r8, r11     ; comparison  [r2 = r8 < r11 ? 1 : 0]
    movn   r8, r11, r2     ; update temporary max. [if (r2 != 0) r8 = r11]
    beqz   r10, end        ; end of the array
    j loop
end:
    sd     r8, max(r0)     ; maximum
```

Debes tratar de entender su funcionamiento. Recuerda que la directiva `.microarch` define la microarquitectura a emplear, por lo que no es necesario modificarla en el simulador.

- Abre el archivo `2-1max.asm` en el simulador.
- Abre el diálogo de microarquitectura y comprueba que la microarquitectura coincide con la especificada después de la directiva `.microarch`. No la cambies, pues en ese caso los cambios introducidos tendrían prioridad sobre los especificados en el archivo fuente.
- Simula instrucción a instrucción al menos dos iteraciones del bucle trazando la ejecución del programa y comprobando los cambios en los registros.
- Una vez termine el programa, ¿qué posiciones de memoria se escribirán? ¿Con qué valores? Responde en el cuestionario: pregunta 3
- Simula hasta el final y comprueba si tu respuesta fue correcta.
- Prueba los botones de navegación del cronograma.

Ejercicios

Como ejercicios propuestos se recomienda llevar los ejemplos de teoría al simulador, escribiendo el archivo ensamblador necesario y configurando la microarquitectura. El simulador puede ser una herramienta muy útil para asimilar los conceptos teóricos.

Apéndice. Juego de instrucciones del simulador pyMIPS64

El simulador pyMIPS64 soporta el siguiente subconjunto de instrucciones de la arquitectura MIPS64. Se corresponde con las instrucciones empleadas en la parte teórica de la asignatura.

Tabla 1. Juego de instrucciones de pyMIPS64

Instrucción	Descripción
`and rd, rs1, rs2`	AND lógico
`andi rd, rs, inm16`	AND lógico con inmediato extendido con ceros
`or rd, rs1, rs2`	OR lógico
`ori rd, rs, inm16`	OR lógico con inmediato extendido con ceros
`xor rd, rs1, rs2`	XOR lógico
`xori rd, rs, inm16`	XOR lógico con inmediato extendido con ceros
`slt rd, rs1, rs2`	Comparación entera con signo (menor que)
`movz rd, rs1, rs2`	Copia condicional de registro (si 0)
`movn rd, rs1, rs2`	Copia condicional de registro (si no 0)
`dadd rd, rs1, rs2`	Suma de dobles palabras, generando excepción si hay desbordamiento
`daddi rd, rs, inm16`	Suma de doble palabra con inmediato extendido con ceros, generando excepción si hay desbordamiento
`dsub rd, rs1, rs2`	Resta de dobles palabras, generando excepción si hay desbordamiento
`dmul rd, rs1, rs2`	Multiplicación de dobles palabras
`ddiv rd, rs1, rs2`	División de dobles palabras
`add.d fd, fs1, fs2`	Suma de reales de precisión doble
`sub.d fd, fs1, fs2`	Resta de reales de precisión doble
`mul.d fd, fs1, fs2`	Multiplicación de reales de precisión doble
`div.d fd, fs1, fs2`	División de reales de precisión doble
`nop`	Instrucción nula
`lb rd, imm16(rs)`	Carga de byte con extesión de signo
`lbu rd, imm16(rs)`	Carga de byte sin extensión de signo
`lh rd, imm16(rs)`	Carga de media palabra con extensión de signo
`lhu rd, imm16(rs)`	Carga de media palabra sin extensión de signo
`lw rd, imm16(rs)`	Carga de palabra con extensión de signo
`lwu rd, imm16(rs)`	Carga de palabra sin extensión de signo
`ld rd, imm16(rs)`	Carga de doble palabra
`l.d fd, imm16(rs)`	Carga de real de presión doble
`sb rs, imm16(ri)`	Almacenamiento de byte
`sh rs, imm16(ri)`	Almacenamiento de media palabra
`sw rs, imm16(ri)`	Almacenamiento de palabra
`sd rs, imm16(ri)`	Almacenamiento de doble palabra
`s.d fs, imm16(ri)`	Almacenamiento de real de presión doble
`beqz rs, label`	Salto si igual a 0
`bnez rs, label`	Salto si distinto de 0
`beq rs1, rs2, label`	Salto si iguales
`bne rs1, rs2, label`	Salto si distintos
`j label`	Salto incondicional